QCon
全球软件开发大会

InfoQ 技术大会

Altum: 蚂蚁集团新一代大模型训练数据处理系统设计与实践

王鑫

蚂蚁集团 AI 数据引擎技术负责人



QCon
全球软件开发大会

InfoQ 技术大会

目录

01 背景与挑战

02 整体方案

03 关键设计 · 性能优化

04 关键设计 · 稳定性提升

05 关键设计 · 研发效率提升

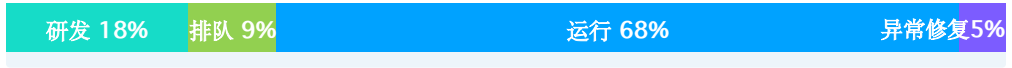
06 生产实践与未来展望

01

背景与挑战

蚂蚁在大模型训练数据处理上面临的挑战

端到端交付耗时占比



性能

1 GPU 算力异构

GPU 型号、资源池与执行环境各异

2 GPU 利用率低

供数与计算衔接不畅，GPU 等待多
碎片化长尾算力闲置突出,算力没有用在刀刃上

3 存算与网络 IO 压力大

逻辑宽表、OSS、数据集全量同步
放大加工开销与数据搬运压力

算力与网络性能压力大,交付易被资源堵住

稳定性

1 任务失败率高

任务/组件易失败，局部异常影响交付

2 链路诊断难

链路跨多个引擎，状态与日志分散
无法串联观测,失败只能逐段排查

3 恢复成本高

缺少细粒度恢复，失败后重复计算
异常从发现到恢复长期依赖人工盯守

失败 → 重跑 → 再排队,交付易被反复拖慢

效率

1 业务多

百灵、阿福、灵光、阿宝、金融等多种业务场景
语言、多模等多种模态

2 模型迭代快

训练节奏牵引数据节奏
数据规则、特征与配比频繁调整

3 经验复用难

算子、流程与调优经验难跨场景复用

平台产能易影响业务迭代速度

核心诉求：提高单位资源有效产出，缩短训练数据交付周期

算力开销、失败重跑与研发等待相互叠加，影响训练数据交付节奏

02

整体方案

Altum 的定位与边界

Altum是一个高性能、高可靠、低成本的大模型训练数据加工系统。



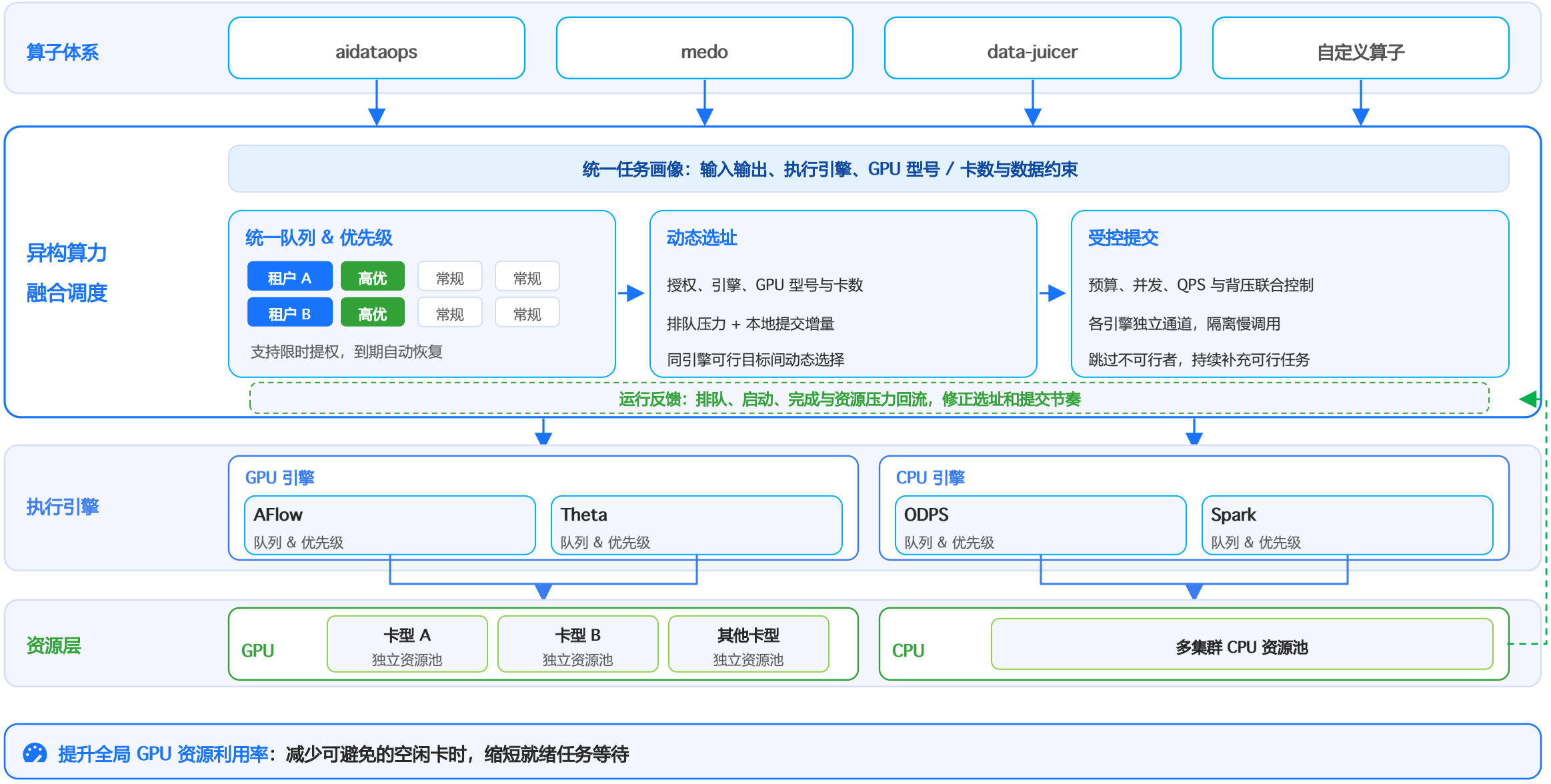
Altum 整体架构


业务层 · 业务场景
训练覆盖预训练 / 后训练 / RL
百灵 · 阿福 · 灵光 · 阿宝 · 金融 · 等
语言模型训练 · 多模态训练 · RAG 数据加工
数据层 · 采集到消费全链路*
Paimon 多模态湖
数据采集
raw data
特征计算
processed data
上架上云
trainable data
训练消费
模型训练 / RAG
研发层 · 研发范式与算子体系
低门槛 · 可复用
① 研发形态 + 研发能力开放
研发形态: GUI · Agent · DIMA
研发能力开放: MCP · Agent · OpenAPI
② Pipeline: 场景化范式 × 能力节点 × 研发算子
场景化范式: 如HF 拓源 · PDF 解析 · 特征计算
能力节点: 如GPU批推 · 去毒 · 质检 · 上云
研发算子*: 如aidataops · medo · data-juicer
运行层 · Pipeline 统一编排&调度(计算 / 控制节点解耦)
异构算力统一调度
基础运维能力: Quota 管理 · 租户隔离 · 智能诊断 · 实例运维 · 数据 trace 等
受控运行时: 端到端可观测 · 异构算力融合调度* · 任务管控 · 快恢 等
引擎层 · 多引擎适配: AFlow/Ray · Theta · 等 | ODPS · Spark · 等
资源层 · 资源统一供给调度(HCS): 多种GPU卡型 · 多机房部署 · 碎片长尾算力整合
资产管治
竖贯全链
资产元数据
资产血缘
资产治理
发布管控
规范校验
风险校验
贯穿数据采集、加工、
交付与消费全流程

03

关键设计 · 性能优化

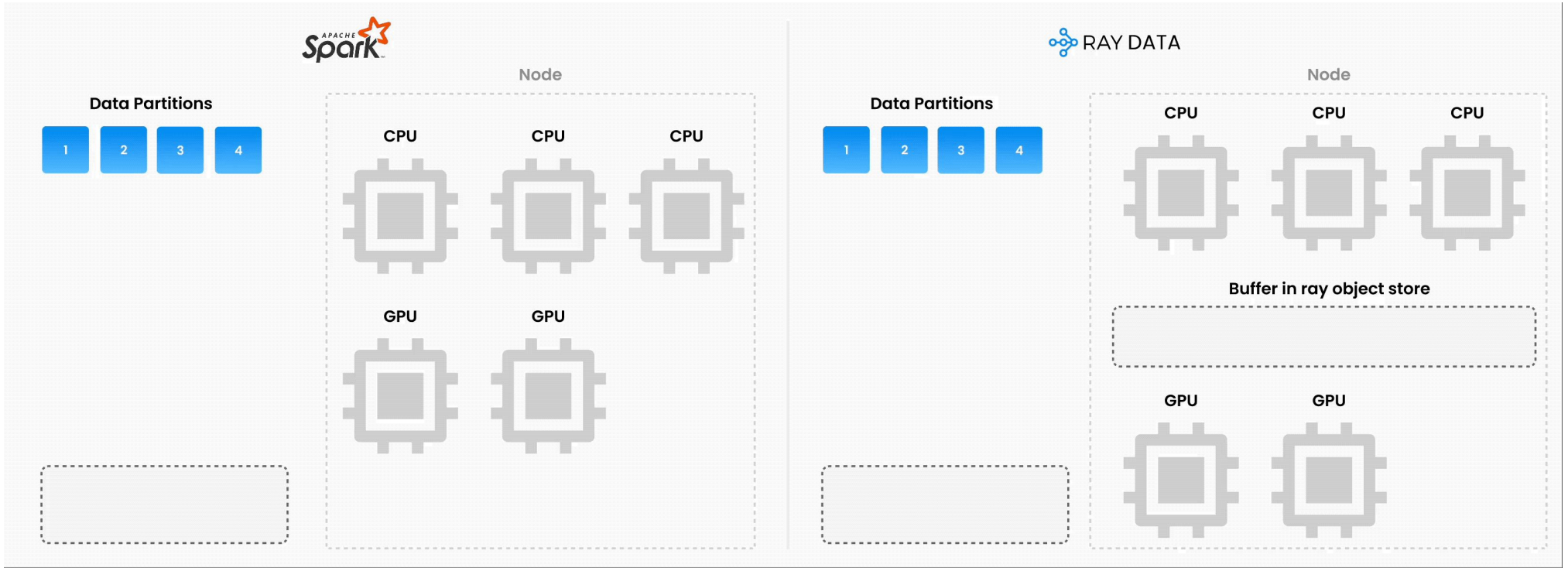
性能优化: Altum 异构算力融合调度(一)



提升全局 GPU 资源利用率：减少可避免的空闲卡时，缩短就绪任务等待

性能优化: Altum 异构算力融合调度(二)

基于AFlow / Ray Data 的流式批处理：数据持续流入,各算子并行消费,无全局屏障,避免GPU大量空转



无 Stage 屏障 · 算子并行处理
上游产出一块,立即派给下游

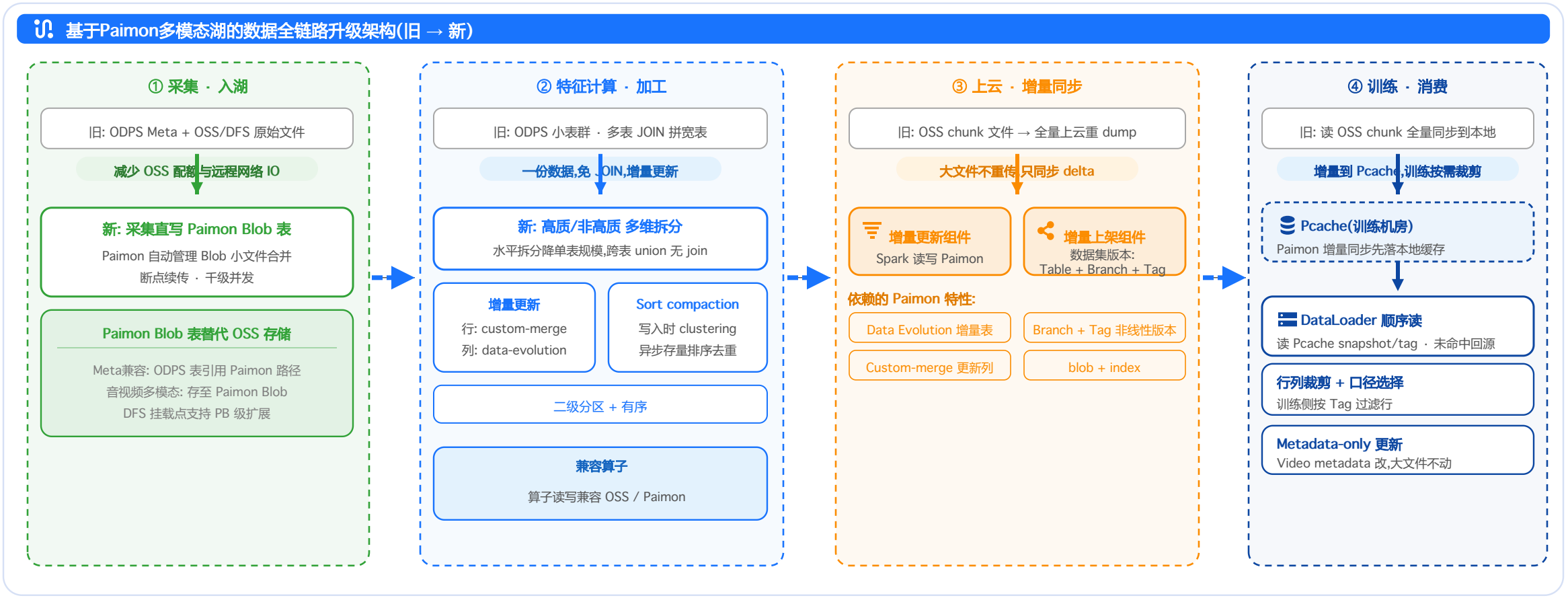
Driver 主动分发 · 算子不直连
算子间只传引用(RefBundle)

AutoScaler · 算子级弹性伸缩
CPU / GPU 各自独立扩缩

GPU原生支持
算子级资源声明,推理常驻

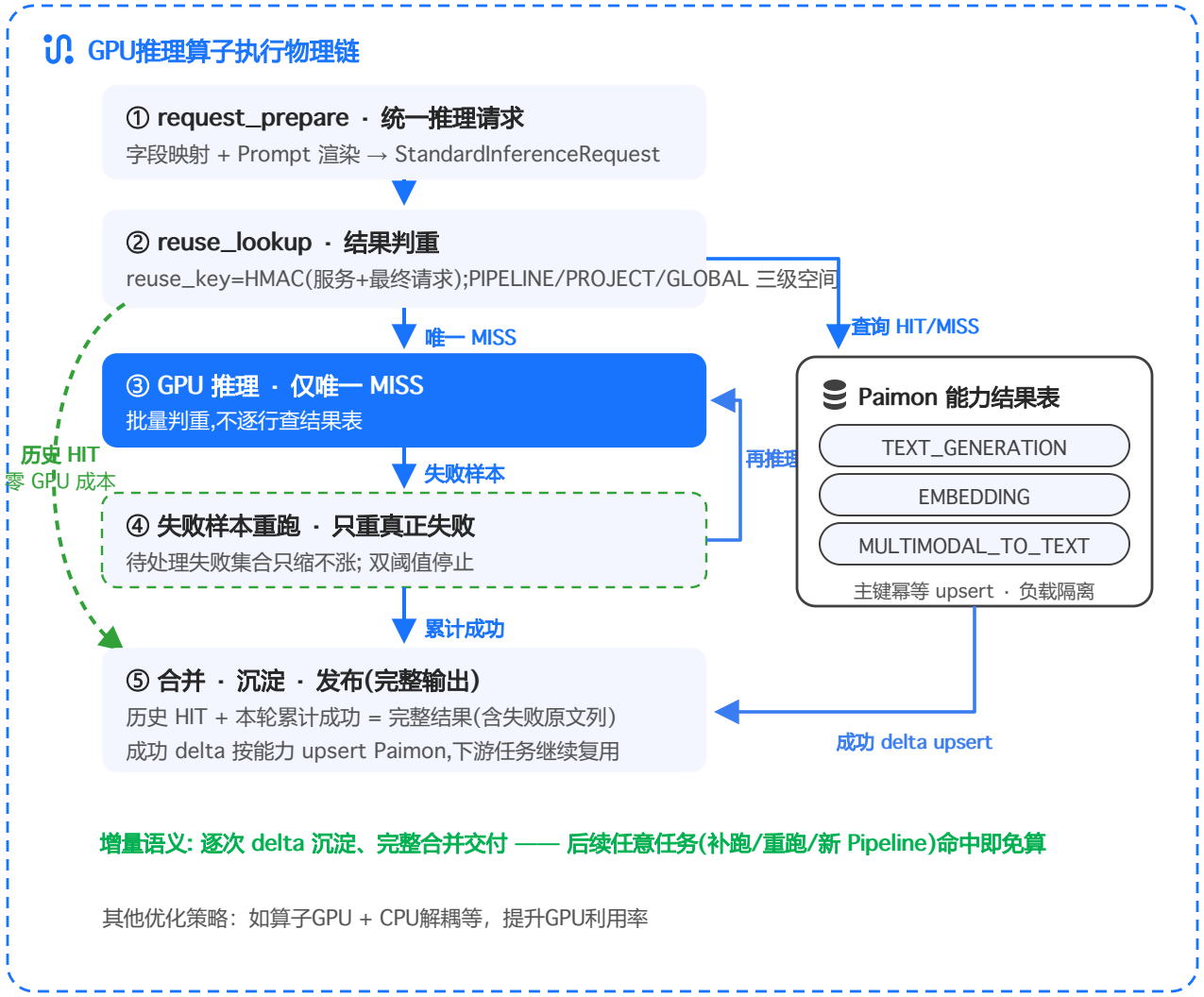
图片来源: Ray E2E Multimodal AI Workloads · Batch Inference(docs.ray.io)

性能优化: 基于多模态湖的端到端升级



升级核心: Paimon Data Evolution + Blob Descriptor + Branch/Tag + Custom-merge + Compaction → 一份数据存储 · 增量更新 · 按需消费

性能优化: Altum 算子计算性能优化



关键设计

① 全局去重

reuse_key 按最终请求精确判重,不凭表主键/场景名猜;
重跑 · 补跑 · 分区重叠 · 人工补数 → 命中即复用

② 样本重试

每轮只提交尚未成功的样本;
行数/比例双阈值停止, 耗尽策略显式声明

③ 增量同步

成功 delta 按能力分表幂等 upsert,逐次沉淀;
输出 = 历史命中 + 本轮成功合并 —— 增量推理,完整交付

以一次数据补跑为例

10 万 输入行 → 8 万 唯一请求 → 命中 5 万 零 GPU 复用 → 3 万 进 GPU

如失败 600 条 → 只重跑这 600 条(而非整批)

一次补跑即节省 62.5% 的 GPU 推理量; 防止 8 万全量重推

GPU 计算量 = 唯一 MISS = 唯一请求 − 复用命中

沉淀得越多,命中越省 —— 增量复用对 GPU 成本产生复利

04

关键设计 · 稳定性提升

稳定性提升: 四道防线的自愈体系

Pipeline 编排 → 节点 → 任务 / 算子粒度

任一级都可能失败: 编排出错 · 节点异常 · 任务排队超时 · 算子执行异常
(随时随地,而非集中在管线尾端)

01 发现

端到端可观测 —— 看清全链路

端到端可观测
贯通 Pipeline、SLA达成全链路可观

进展与资源监测
识别卡顿、反压与资源瓶颈,异常即刻暴露

产物与交付校验
确认产物存在、版本正确,并完成下游交接

异常告警与追踪
按对象与执行记录定位,告警可下钻、可归并

02 防护

熔断与资源保护 —— 不让局部故障放大到全局

熔断与探测恢复
持续异常先暂停加压,冷却后探测,半开缓升温

限流与并发控制
防止过载与重试风暴,重查询不压垮轻量信号

资源隔离与配额
限制租户与任务资源占用,租户/队列互不挤兑

重试预算与准入
限制轮次、时限与恢复范围,防无效重试烧资源

03 诊断

智能诊断 —— 结论优先,证据链可追溯

智能故障归因
分析错误模式与执行上下文,结论先行展开证据

多源证据关联
关联日志、指标、配置与血缘,逐 Job 取证

依赖链路排查
定位上下游依赖与失败传播,看清影响面

处置建议与知识沉淀
给出恢复建议,复用故障案例,经验进知识层

04 自愈

样本级重试与恢复策略 —— 保留成功,只愈失败

样本级重试
保留成功结果,只重试失败样本,失败集只缩不涨

断点续跑
从检查点恢复,减少整链重跑,范围明确可回查

分级恢复策略
按样本、节点、任务粒度选择恢复级别与动作

恢复校验与兜底
确认产物一致;幂等/轮次/时限/预算约束,超限转人工

恢复结果回流
校准规则与策略

05

关键设计 · 研发效率提升

研发效率提升: 端到端场景化 Pipeline 研发

人机协同研发

人把关方向,Agent 跑执行

- 人(Owner): 定口径 · 明确业务含义 · 审核结果
- Agent: 执行 · 分析 · 记录
- Research Agent: 自主提假设(长期)

场景化范式

同类场景 → 沉淀为可复用范式而非一次性脚本

- 特征计算范式
- 网页/PDF解析范式
- 拓源范式
- 自定义 范式

能力节点(基础节点 + 增强节点) + Pipeline 编排

节点可插拔 · DAG 动态调整

基础节点

- 通用Python
- 通用 SQL
- ...

增强节点

- GPU批量推理
- 质检 · 去毒
- 特征计算
- Agentic 逐条处理
- 增量上云
- Token 化

算子体系

标准化 · 管理体系

- 官方算子库
- 算子标准(兼容三方): aidataops / medo / data-juicer
- 算子研发管理体系

Paimon 多模态湖 · 全量底座

rawdata 全量在湖 · 稳定样本唯一 ID · 处理全打标 · 特征口径、版本、出湖记录、实验结果全链可追溯

一次打标 · 多口径出湖

范式定义"做什么" → 节点决定"用什么零件" → 算子提供"最小加工单元"—— 层层下沉,开发只需组合,不改内核

复杂能力预先内置在增强节点与官方算子库中,新链路研发从「写 ETL 脚本」变为「选范式 + 配节点 + 调参数」

研发效率提升: 场景化研发范式 - 特征计算范式

不用范式 —— 散表 + 固定 ETL

① 建表 + 写 ETL
ODPS 建子表 → 写 SQL 拼宽表 → 维护多表间关系

② 改口径 → 整表重跑
insert overwrite;改阈值 = 重跑全链;改一列 = 重建所有 JOIN

③ 全量同步(每次都全传)
改配比 → 重 dump → 跨机房全量重传;无法按需消费

④ 上架输出(一次性交付)
交付完即结束;结果不回流;历史解释不了;经验散在脚本里

特征不是资产,只是表里的列:
无 Owner · 无口径版本 · 无实验记录 · 无血缘 · 不可跨业务复用

用范式 —— 多模态湖 + 特征口径持续迭代

Paimon 全量底座:唯一 ID(_ai_unique_id_) · rawdata 全留 · 处理仅追加标签 · 选择在出湖做

1. **数据入湖 + 定义特征口径**
   唯一 ID 校验补齐 · 填写业务含义 / Owner / 计算逻辑 · 表准入(分区键 · 唯一键 · data-evolution 模式)
2. **样本调试 → 全量特征计算(增量更新)**
   小样本前后对照 · self-merge 新增/更新列 · 增量行刷新(不重算已有) · 断点续跑 · 并行 Job / DAG
3. **特征分布查看 → 选择口径出湖**
   数值分桶 · 类别频次 · 覆盖率 · Btree/BitMap 索引;多口径 ≡ 一个数据集(保留 URL 高质/非高质/HOST 三表分组)
4. **上架消费 + 消融实验**
   训练侧按口径/版本读取 · 行列裁剪 · 增量上云只同步 delta;消融实验记录基线/变体 · 关联数据集版本+特征版本
5. **实验结果回流 → 驱动下一批数据迭代(飞轮闭环,非一次交付)**

技术保障:

- 增量更新 · self-merge
- 水平拆表 · 超阈值裂变
- clustering 写入排序
- 异步 sort compaction
- host 一级分区/url 二级
- 分布式分区 compaction
- Btree · BitMap 索引
- 血缘 · TraceID 全链路

一份数据 + 按口径选择,增量更新只算 delta; 特征具有 Owner/口径/版本/实验记录,是可管理资产
能发现(湖表+特征清单) · 能理解(分布+口径) · 能追溯(血缘+版本+实验) · 能使用(多口径出湖上架)

研发效率提升: 训练数据处理 - Agentic逐条模式 vs 传统模式

传统模式 · 数据集级流水线

数据集(整批) → 按类型路由 → 固定 Pipeline (清洗 · 质检(标准流水线)) → 固定准出规则 (判定 → 上架输出)

整批处理 · 一套固定规则覆盖整个数据集 · 无法逐条感知特征

升级:粒度下沉到单条 · 判断让位 Agent · 工具串联

逐条处理模式 · 单条级 Agent

ODPS 数据表 → 按行拆分 → UDF 入口 (逐条调用) → Claude Agent (多轮迭代 · 上下文压缩) → 模型自判达标 (写回目标表)

工具多轮迭代

数据/样例要求 / 资源要求随单条读取,单条全流程闭环;UDF 归集检查结果,分布式写回

- 分布式执行:ODPS / Spark
- 沙箱隔离
- 滑动窗口 · 多并发
- Session 即用即销

逐条并行 × 分布式执行 —— 大规模数据下的 Agent 清洗吞吐

对比维度	传统模式 数据集级 · 固定 Pipeline	逐条处理模式 单条级 · Agent + Claude
处理粒度	**数据集级** 按大类整体处理	**单条级** 按每条数据特征检测是否合格
判断方式	**固定准出规则** 标准流水线统一判定	**Agent 架构** 模型自主判断是否达标
驱动机制	规则 + 固定 Pipeline 驱动	LLM + 工具 **多轮迭代** 调用
复杂操作	**强依赖外部工具** 单靠 LLM 难独立完成	**Claude 串联工具** 一次完成格式/含毒/中英混杂等
长程稳定	**无状态规则** 复杂任务易中断	Claude **上下文压缩** 长程清洗稳定高效
执行架构	**固定 Pipeline** 各自维护工程,成本高	UDF 调 Claude Code · ODPS/Spark 分布式 · 滑动窗口 + **Session 即用即销**

清洗逻辑即 Prompt:输出按 JSON schema 定义,沉淀模板跨业务复用

粒度下沉 + 判断让位 Agent + 工具串联 —— 规则工程转向 Agent 工程

应用效果

4B ≈ 120B

mid-train 场景 · 同等数据效果

mid-train场景: 逐条处理模式 + 4B 模型 ≈ 传统模式 + 120B 模型

质量来自智能数据管道 —— 管道升级换模型降档,推理成本与显存大幅下降

06

生产实践与未来展望

Altum 在蚂蚁业务场景的落地效果

应用案例 · 数据飞轮场景 —— 编码助手Session轨迹多标签批量打标

千万级

打标规模

交付窗口内完成多标签分类打标
效果稳定,结果全链可追溯

>99%

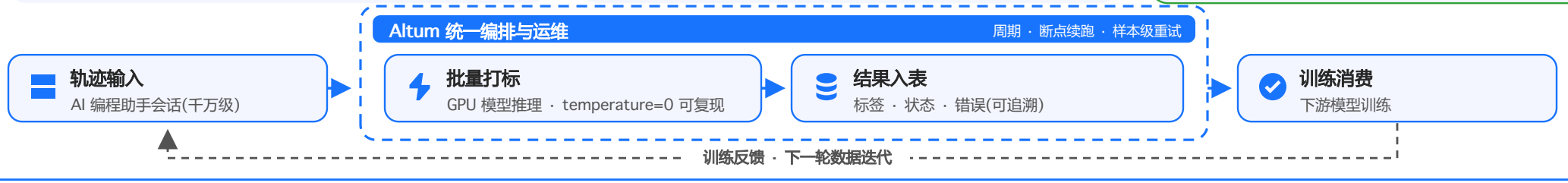
打标成功率

较之前同规模任务执行成功率大幅提升

96%

同配置复跑, 标签完全一致（Exact Match）率

差异 ≤1 个标签的样本占 99%,重跑抖动可控



性能：提高单位算力产出

- **GPU计算性能提升：**多模态算子 GPU 与 CPU 解耦&统一调度，GPU 利用率3~5x提升
- **IO性能提升：**基于多模态湖，端到端数据存储成本大幅下降，IO性能1~5x提升

稳定性：保障生产数据持续交付

- **异常定位有据可查：**关联任务状态、执行记录与样本错误，支持故障追溯
- **失败恢复成本降低：**通过断点续跑、节点重试，减少整链重跑与重复计算，失败率大幅下降

效率：加快业务接入与数据迭代

- **业务接入与交付迭代更快：**AI + 场景化范式开箱即用，研发效率大幅提升
- **研发经验可复用：**沉淀标准算子与场景化 Pipeline，减少跨业务重复研发

规模化应用: 百灵、阿福、灵光、阿宝、金融等场景已常态化生产运行

把大模型数据加工从「一次性脚本」升级为「标准化 · 可恢复 · 可复用 · 可观测」的工业化数据生产能力。

AI Data Infra 演进方向: 一条线 → 一个环 → AI 自主的环

人工 · 工业化流水线 —— 一条线



单向开环 · 每个节点由人推动 · 交付即结束

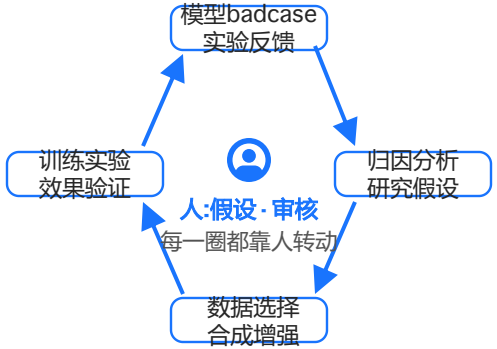
一次成线: 人工定义流程，按规则生产与交付

标准算子与稳定批量处理
全链由人驱动,人盯每个节点
规则与配比即配置,交付即终点
批量重复劳动,经验只沉为脚本与分数

交付物 DATASET · 度量:数据量与质量

行业相对成熟

人机协作 · 模型badcase/实验反馈闭环 —— 一个环



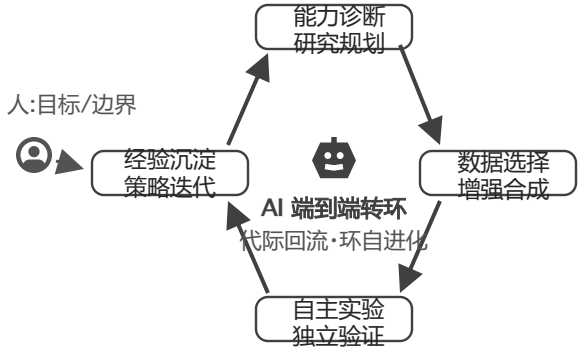
反馈成环: 人机分析反馈，迭代数据方案

能力诊断、对照实验与数据迭代
Evaluation 驱动下一批数据
关键判断由人把关: 假设·审核
Trace 沉淀经验,人工确认后入库复用

交付物 能力导向数据 · 度量:消融增益

现状

RSI · AI 自主研究完成闭环全流程(自进化)



RSI: 人设目标与边界，AI 驱动全流程

动态规划、经验记忆与改进继承
人只出现在两处: 设定目标边界,裁决继承
中心瓶颈: 安全继承证据
评价体系随研究一起审计迭代

交付物 进化系统 · 度量:可继承证据

探索中 · RSI Agent

两个跨越: ① 线 → 环: 把「交付终点」变成「反馈起点」② 人环 → AI 环: 评价独立化 + 证据协议 + 预算护栏

以实验与模型反馈驱动迭代，再由 AI 自主规划、执行与验证，持续沉淀可复用经验

AiCon 北京站

全球人工智能开发与应用大会

探索 AI 应用边界

Explore the limits of AI applications

2026 年 12 月 18–19 日 / 北京新云南皇冠假日酒店

参会咨询

专题：Post-Training Engineering | 数据、奖励与模型能力塑造



专题出品人：余瑶

飞猪 / 资深算法专家

专题：大模型推理与算力调度实践



专题出品人：范苑

阿里云 / 专有云计算产品负责人

专题：Knowledge Engineering | 让 Agent 理解企业



专题出品人：盛茂家

火山引擎 / Viking 团队技术负责人

专题：Loop & Graph Engineering



专题出品人：孙子荀

腾讯 / 腾讯高级总监

专题：AI 原生架构的演进与治理



专题出品人：刘中兵

快手 / 架构治理团队 & AI 架构演进负责人

专题：Agentic Software Engineering | 从代码生成到软件交付



专题出品人：杨杰

荣耀 / 首席 IT 技术架构师

QCon
全球软件开发大会

InfoQ 技术大会

THANKS

大模型正在重新定义软件

Large Language Model Is Redefining The Software

